# From Choices to Sums

Connections to OpenAI's Subset-Sum Algorithms

Jonathan R. Landers October 28, 2026

Expository research note · Working draft

A list of $n$ numbers has $2^n$ subsets. Many subsets can ask the same question, and one mathematical object can answer many questions at once. Distinct sums and reusable representations connect this research sequence to OpenAI's recent subset-sum manuscripts. They reveal how the structure of a search determines the work needed to resolve it.

## An exponential search with only two questions

Given positive integer weights $w_1, \ldots, w_n$ and a target $t$, subset sum asks whether some selection adds to $t$. Each listed item can be used once. Consider the weights $[2, 5, 6, 11, 23]$ and target 17. Set aside 6. The remaining weights $[2, 5, 11, 23]$ are *superincreasing*: each exceeds the sum of all its predecessors. Descending greedy choice decides membership in this core. Leaving 6 out fails; taking it leaves 11, which the core reaches. Two core queries resolve the original 32 choices.

Note V [1] makes this construction systematic. Sorting and a greedy scan find a largest superincreasing core $C$ in $O(n \log(n+1))$ arithmetic and comparison operations. Let $E$ contain its $e$ exceptional items. Each subset of $E$ supplies a shifted target for $C$. Equal exceptional sums supply equal queries, so the relevant quantity is

$$\mathcal{A}(E) = \left\{ \sum_{i \in A} w_i : A \subseteq E \right\}, \qquad K = |\mathcal{A}(E)|.$$

This set of reachable sums is the *support*. Build it by repeatedly merging a sorted list with a shifted copy, keeping one indexed witness per sum. Every intermediate support is contained in the final support: later items can always be left out. Construction takes $O(eK)$ operations; the $K$ greedy queries take $O(nK)$. For total binary input length $L$, the resulting bit-time bound is $\mathrm{poly}(L)K$.

## A shared measure of compression

OpenAI's low-space manuscript [2] measures a block $V$ through its *deficiency*,

$$d(V) = |V| - \log_2 |\mathcal{A}(V)|.$$

Applying this definition to Note V rewrites its existing runtime bound as

$$\boxed{T_V \leq \mathrm{poly}(L)\, 2^{e - d(E)}.}$$

The formula identifies two sources of savings: fewer exceptional items and more merging among their sums. Twelve equal positive exceptions have only 13 sums, giving $d(E) = 12 - \log_2 13$. Their 4,096 selections require just 13 core queries. The exponent counts the distinct questions still to answer.

## Few sums compress; many sums support filtering

The low-space manuscript uses deficiency to analyze two complementary randomized trials [2]. One compresses repeated sums in disjoint blocks and combines the resulting lists through sorted streams. The other uses blocks with many distinct sums to give a solution many decompositions, then filters them by requiring chosen remainders modulo integers. Each round runs both trials; their analysis uses the mean deficiency of a random block.

This complementary strategy belongs to a line of work developed by Austrin, Kaski, Koivisto and Nederlof [3]. It makes distinct-sum structure useful on both sides: concentrated sums favor compression, while abundant sums can sustain many representations through filtering. OpenAI's September manuscript states $\mathrm{poly}(n)2^{n/2}$ time and ordinary $O(2^{n/5})$ writable-word space for polynomial-bit inputs. Its guarantees include discovery, generation and storage costs, with one-sided error and success probability at least $2/3$ on each solvable input [2].

The representations retain the information needed to combine them. A sum of 5 could use the item 5 or the items 2 and 3. To combine that record with the item 2, we must know which indices it uses: only the first choice gives a valid subset of sum 7. OpenAI's overlapping constructions therefore retain membership masks alongside weights. Note V combines disjoint index sets, so one witness per exceptional sum supplies all the information its core queries need. This gives a useful design rule: merge records when they have the same valid continuations.

## A checksum that recovers the exact target

The October manuscript [4] states a randomized worst-case bound of $O(2^{0.49n})$ word operations for each fixed polynomial bound on input bit length, with $O(n+b)$-bit words when the maximum input bit length is $b$. It gives correctness probability at least $2/3$ on each fixed input and caps time on every execution. The exponent improves Horowitz and Sahni's 1974 meet-in-the-middle scale $2^{n/2}$: their exponential factors have ratio $2^{0.01n}$.

A preliminary test handles small partial-sum supports. The remaining method isolates a solution using random tie-breaking weights. For transformed weights $c_i$, target $t'$ and prime $P$, let $c(I) = \sum_{i \in I} c_i$ and assign each subset a random complex phase $\chi(I)$ of absolute value one. Its organizing identity is

$$\underbrace{\sum_{c(I)=t'} \chi(I)}_{\text{exact target}} = \underbrace{\sum_{c(I) \equiv t' \pmod P} \chi(I)}_{\text{modular checksum}} - \underbrace{\sum_{\substack{c(I) \equiv t' \pmod P \\ c(I) \neq t'}} \chi(I)}_{\text{aliases}}.$$

A Fourier formula expresses the checksum as an average of products built from individual weights. Filters organize which terms to evaluate or sample; compatible representations recover the aliases. After subtraction, an empty target has value zero and an isolated solution has absolute value one. Absolute error below $1/4$ permits detection at threshold $1/2$. The method turns a large collection of subsets into an aggregate whose surviving value answers the exact question [4].

## A geometric representation that answers many targets

Note VII [5] approaches aggregation through a discovered base $M \geq 2$. After exact reductions, it writes the positive weights as

$$u_i = Mp_i + r_i, \qquad p_i \geq 0,$$

selects a core $C$ with a narrow residual range, and sets aside exceptions $E$. Each core subset becomes a point

$$(q, s) = \left( \sum_{i \in C} p_i x_i,\ \sum_{i \in C} r_i x_i \right).$$

An exact dynamic program stores reachable points. For each exceptional assignment $A$, the target line is

$$Mq + s = z_A, \qquad z_A = T - \sum_{i \in A} u_i.$$

The geometry stays fixed as the line moves. With core $[101, 203]$, exception 149, base $M = 100$ and target 250, taking the exception shifts the line to $100q + s = 101$. The reachable point $(1, 1)$ supplies the core item 101. The original solution is $149 + 101 = 250$.

The shared table lies in a rectangle. Write

$$R_- = \sum_{i \in C} \min(0, r_i), \quad R_+ = \sum_{i \in C} \max(0, r_i), \quad W = R_+ - R_- + 1,$$

$$Q = \min \left( \sum_{i \in C} p_i, \left\lfloor \frac{T - R_-}{M} \right\rfloor \right), \qquad H = Q + 1.$$

For $Q \geq 0$, these bounds cover every shifted solution: positive exceptional weights give $z_A \leq T$, and $s \geq R_-$ bounds the required quotient. The nonnegative $p_i$ make the quotient cap safe during construction. A query tests at most $G$ points where its line crosses the rectangle, with $1 \leq G \leq H$.

Let $F$ pay reductions and discovery. The runtime companion [6] gives the conservative bit-time envelope

$$T_{\text{geom}} \leq F + O^*(HW + 2^e GW),$$

where $O^*$ suppresses factors polynomial in $L$. The terms expose the benefit: one table construction serves all exceptional assignments. A table of $HW$ positions can summarize exponentially many core subsets. Polynomial discovery, polynomial $HW$ and $e = O(\log L)$ give a polynomial regime.

Note III [7] supplies another view of aggregation. If $r(t)$ counts representations of a target by two half-sum sets, put

$$B = \sum_{t \in D} r(t), \quad J = \sum_{t \in D} \binom{r(t)}{2}, \qquad \sum_{t \in D} r(t)^2 = B + 2J.$$

For $B > 0$, Cauchy–Schwarz gives at least $B^2/(B + 2J)$ reachable targets in $D$. Multiplicities become a certificate about distinct outcomes. Across these constructions, counts, coordinates, masks and phases each preserve the information required by the question being asked.

## Combining the two structural savings

The distinct-sum and shared-table constructions fit together directly. First discover the geometric core. Then construct the $K = |\mathcal{A}(E)|$ distinct exceptional sums, retaining a witness for each. Query the common table once per surviving shifted target. Equal sums have equal residual targets, and their witnesses lie entirely outside the core.

**Derived connection.** Combining Note V's support construction with Note VII's shared table gives an exact algorithm with the bit-time bound

$$\boxed{T_{\text{combined}} \le F + O^*\Big(HW + 2^{e-d(E)}(1+GW)\Big)\,.}$$

The support construction costs $O^*(K)$; bounds and table queries cost $O^*(K(1+GW))$. One core table costs $O^*(HW)$. Prefix history and exceptional witnesses occupy $O^*(HW+K)$ bits.

This derived algorithm combines three measurable opportunities: a small core table, few distinct exceptional sums and short line queries. For polynomial-bit inputs, its bound lies below the $2^{0.49n}$ scale when, for a fixed $\epsilon > 0$, discovery takes $O^*(2^{(0.49-\epsilon)n})$ and

$$\max\left\{\log_2(HW),\ e - d(E) + \log_2(1+GW)\right\} \le (0.49-\epsilon)n.$$

The exponent margin absorbs polynomial encoding costs. This condition turns the connection into a concrete question about which input families offer cheaply discoverable compression and reuse.

The repository's measurements show why the full cost matters. In Note V, two selectors reduced support on 18 of 54 structured lists, while their median total times were about nine times those of the simple core solver. In Note VII's first frozen study, reusable cores raised exact completions from 453 to 469 of 576 fresh targets under the same work allowance; the net gain had a whole-list-bootstrap 95% interval of $[4, 30]$. A later matched ablation avoided 90 of 182 table builds by testing necessary bounds first. These results locate useful savings in both representation and execution.

The connection is significant because it gives the research sequence a precise place in the mathematics of subset sum. Distinct support measures how many questions remain. Compatibility determines which records can be merged. Shared tables spread construction cost across queries. OpenAI's manuscripts put compression and filtered representations into worst-case algorithms; the repository makes structural savings explicit and measurable. The common research question is sharp: *which information lets us resolve many choices together, and how cheaply can we obtain and use it?*

## References


[1] J. R. Landers. *How Far from Easy? Structural Distance in Subset Sum*. Note V, working draft, 2026.
[2] OpenAI. *A Low-Space Algorithm for Worst-Case Subset Sum*. Preprint, September 26, 2026.
[3] P. Austrin, P. Kaski, M. Koivisto and J. Nederlof. Dense Subset Sum May Be the Hardest. *STACS*, 13:1–13:14, 2016.
[4] OpenAI. *Subset Sum in Time $O(2^{0.49n})$*. Preprint, October 4, 2026.
[5] J. R. Landers. *Geometric Cores and Exceptions in Subset Sum*. Note VII, working draft, 2026.
[6] J. R. Landers. *The Runtime of Geometric Core Subset Sum*. Runtime companion to Note VII, working draft, 2026.
[7] J. R. Landers. *Probability-Valued Subset Sum: Certified Bounds and Learned Guidance*. Note III, 2026.